Java Constants – Use Constants for Fixed Business Rules

Learning Objectives

After this topic, you will be able to:

- Understand what constants are.
- Learn why constants are used for fixed business rules.
- Create constants using final and static final.
- Follow Java naming conventions.
- Understand the difference between variables and constants.
- Answer common interview questions.

---

1. What is a Constant?

A **constant** is a variable whose value **cannot be changed** after it is assigned.

In Java, constants are created using the **final** keyword.

Example:

```
final double PI = 3.14159;
```

Once assigned, PI cannot be modified.

```
PI = 3.14;   // Compiler Error
```

---

2. Why Use Constants?

Many applications have **fixed business rules** that rarely change.

Instead of writing the same value multiple times (**magic numbers**), define it once as a constant.

Examples:

- Minimum age = 18
- Maximum login attempts = 3
- GST = 18%
- Company name
- API timeout
- Maximum file size

This makes the code:

- Easier to read
- Easier to maintain
- Less error-prone

---

3. Business Rule Example

❌ Without Constants

```
if (age >= 18) {
    System.out.println("Eligible");
}

if (customerAge >= 18) {
    System.out.println("Can Register");
}
```

What if the minimum age changes from **18** to **21**?

You must update every occurrence.

✅ With Constants

```
final int MINIMUM_AGE = 18;

if (age >= MINIMUM_AGE) {
    System.out.println("Eligible");
```

```
}

if (customerAge >= MINIMUM_AGE) {
    System.out.println("Can Register");
}
```

Now you only change one line.

---

4. final Keyword

The final keyword prevents reassignment.

```
final int DAYS_IN_WEEK = 7;
```

Valid:

```
System.out.println(DAYS_IN_WEEK);
```

Invalid:

```
DAYS_IN_WEEK = 8;
```

Compiler Error.

---

5. static final Constants

If a constant belongs to the entire class (shared by all objects), use:

```
public static final int MAX_LOGIN_ATTEMPTS = 3;
```

This is the most common way to declare constants.

Example:

```
public class AppConfig {

    public static final int MAX_LOGIN_ATTEMPTS = 3;
    public static final double GST_RATE = 0.18;
}
```

Usage:

```
System.out.println(AppConfig.MAX_LOGIN_ATTEMPTS);
```

---

6. final vs static final

final	static final
Belongs to an object	Belongs to the class
Each object has its own constant	One shared constant for all objects
Used for instance-specific values	Used for application-wide constants

Example:

```
class Student {

    final int rollNumber;

    Student(int rollNumber) {
        this.rollNumber = rollNumber;
    }
}
```

Each student has a different (but unchangeable) roll number.

---

7. Java Naming Convention

Constants are written in **UPPER_CASE** with words separated by underscores.

✅ Good

```
MAX_USERS
MINIMUM_AGE
PI
MAX_LOGIN_ATTEMPTS
DATABASE_URL
```

❌ Bad

```
maximumAge
MaxAge
maxLoginAttempts
```

---

8. Avoid Magic Numbers

A **magic number** is a hardcoded value whose meaning is unclear.

❌ Bad

```
if (speed > 120) {
    System.out.println("Overspeeding");
}
```

What does 120 represent?

✅ Good

```
final int MAX_SPEED = 120;

if (speed > MAX_SPEED) {
```

```
    System.out.println("Overspeeding");
}
```

Now the code explains itself.

---

9. Real-World Examples

Banking

```
public static final double MIN_BALANCE = 500.0;
```

Login System

```
public static final int MAX_LOGIN_ATTEMPTS = 3;
```

E-commerce

```
public static final double TAX_RATE = 0.18;
```

File Upload

```
public static final int MAX_FILE_SIZE_MB = 10;
```

---

10. Constants and Objects

final behaves differently for primitive types and reference types.

Primitive

```
final int x = 10;
```

Cannot change the value.

Reference Type

```
final StringBuilder sb = new StringBuilder("Java");
```

You **cannot reassign** the reference:

```
sb = new StringBuilder("Python"); // Error
```

But you **can modify the object**:

```
sb.append(" Programming");
```

Important Interview Point: final prevents changing the **reference**, not necessarily the **object** (unless the object itself is immutable, like String).

---

11. Best Practices

- Use constants for values that should never change.
- Replace magic numbers with meaningful constant names.
- Use public static final for shared application constants.
- Follow the UPPER_CASE naming convention.
- Group related constants in a separate configuration or constants class.

---

12. Common Mistakes

❌ Hardcoding values everywhere.

```
if (age >= 18)
```

Repeated throughout the project.

❌ Naming constants like normal variables.

```
final int minimumAge = 18;
```

Prefer:

```
final int MINIMUM_AGE = 18;
```

❌ Assuming final makes every object immutable.

final only prevents reassignment of the reference.

---

13. Tricky Interview Questions

Q1. What is a constant?

Answer:

A variable whose value cannot be changed after initialization.

Q2. Which keyword is used to create constants?

Answer:

final

Q3. Why should we use constants instead of magic numbers?

Answer:

- Improves readability
- Easier maintenance

- Prevents accidental changes
- Centralizes business rules

Q4. What is the difference between final and static final?

Answer:

- final → Object-level constant.
- static final → Class-level constant shared by all objects.

Q5. Why are constants written in uppercase?

Answer:

It is the standard Java naming convention and makes constants easy to identify.

Q6. Can a final variable be assigned twice?

```
final int x;
x = 10;
x = 20;
```

Answer:

No. A final variable can be assigned **only once**.

Q7. Is String a constant because it's immutable?

Answer:

No. A String object is immutable, but a String variable is not automatically a constant.

```
String name = "Java";
name = "Python";   // Valid
```

To make the reference constant:

```
final String name = "Java";
```

Q8. Does final mean compile-time constant?

Answer:

Not always.

```
final int x = 10;
```

This is a compile-time constant.

But:

```
final int age = scanner.nextInt();
```

This is assigned at runtime, so it is **not** a compile-time constant.

Q9. Can a final object change?

Answer:

Yes, if the object is mutable.

```
final ArrayList list = new ArrayList<>();

list.add("Java"); // Allowed
```

The reference cannot change, but the object's contents can.

Q10. Where should application-wide constants be stored?

Answer:

In a dedicated constants or configuration class using public static final.

---

14. Output-Based Questions

Question 1

```
final int MAX = 100;

System.out.println(MAX);
```

Output

```
100
```

Question 2

```
final int MAX = 100;

MAX = 200;
```

Output

```
Compiler Error
```

Question 3

```
final StringBuilder sb = new StringBuilder("Java");

sb.append(" Programming");

System.out.println(sb);
```

Output

```
Java Programming
```

Question 4

```
final StringBuilder sb = new StringBuilder("Java");

sb = new StringBuilder("Python");
```

Output

```
Compiler Error
```

---

15. Cheat Sheet

Keyword	Purpose
final	Prevents reassignment
static	Belongs to the class
static final	Shared class constant

Naming Convention

```
MAX_LOGIN_ATTEMPTS
MINIMUM_AGE
TAX_RATE
PI
```

Key Points to Remember

- A **constant** is a value that should not change.
- Use the **final** keyword to create constants.
- Use **static final** for shared application-wide constants.

- Avoid **magic numbers** by replacing them with meaningful constant names.
- Write constants in **UPPER_CASE_WITH_UNDERSCORES**.
- final prevents **reassignment**; it does **not** automatically make mutable objects immutable.

> **Interview Tip:** One of the most common Java interview questions is:
>
> **"What is the difference between final, finally, and finalize()?"**
>
> - **final** → Keyword used to restrict modification (variables, methods, classes).
> - **finally** → Block used with exception handling that usually executes regardless of whether an exception occurs.
> - **finalize()** → A method that was associated with garbage collection and is **deprecated**; it should not be relied upon in modern Java.